

SWYNEX-AI-Problem-Design

Task 1: AI Problem Design | SWYNEX
Technologies Internship

Problem Statement

Sellers on a local marketplace often post listings with the wrong category (or none), which makes items hard to find. This project defines an AI **text classification** system that reads a listing's title and description and assigns it the correct category automatically.

Use Case

Task type: Text classification (single label)

Example

Input (title + description)	Predicted category
"iPhone 12, 128GB, good condition, with box"	Mobiles & Electronics
"Sofa set 3+1, teak wood, 2 years old"	Furniture
"Royal Enfield Classic 350, 2019, 22,000 km"	Vehicles

Categories (8): Mobiles & Electronics, Furniture, Vehicles, Fashion, Home Appliances, Books & Education, Sports & Fitness, Others

User

- **Primary user:** A seller posting an item on a local marketplace website (for example, a Hyderabad-based platform).
- **Secondary user:** The platform admin, who wants clean, correctly categorized listings without manual review.

Data Source

- **Training/test data:** A small hand-built dataset of about 200 listings (25 per category), written by me in the style of real local classifieds, with variations in wording, abbreviations, and Indian price and brand conventions.
- **Split:** 50 listings held out as a test set (never used for prompt tuning or examples).
- **Format:** CSV with columns `title`, `description`, `true_category`.
- **Model approach:** A pre-trained LLM prompted with the category list and a few examples (few-shot), no custom training required.

Constraints

- **Size:** Small dataset (about 200 rows), so the system must work with few-shot prompting rather than heavy training.
- **Cost:** Must stay low-cost, ideally a fraction of a rupee per listing.
- **Speed:** Under 3 seconds per listing so it feels instant when posting.
- **Privacy:** No phone numbers, addresses, or personal details sent to the model. Listings are cleaned first.

- **Language:** English, with common Hinglish or Telugu-English words (for example "purana", "ek saal purana").

Success Criteria

The system is considered successful if it meets **all** of these on the 50-listing test set:

Metric	Target
Overall accuracy	at least 85%
Per-category recall	at least 70% for every category
Invalid outputs (label not in list)	0%
Average response time	under 3 seconds

Evaluation Approach

1. Run the classifier on all 50 held-out test listings.
2. Compare predictions with `true_category` .

3. Calculate accuracy, per-category precision and recall, and a confusion matrix.
4. Review every wrong prediction and group the errors (for example, "Sports vs Fashion for shoes").
5. Improve the prompt or examples, then re-run on the **same** test set and report before and after results.

Risks and Limitations

- Ambiguous listings (for example, "cycle for kids", which could be Sports or Others).
- Mixed-language text may lower accuracy.
- A small test set means results are indicative, not a guarantee.
- Uncertain predictions should fall back to "Others" or be flagged for manual review.

Repository Structure

```
SWYNEX-AI-Problem-Design/  
├── README.md           <- this  
document  
├── data/  
|   └── sample_listings.csv
```

```
(optional: sample of the dataset)
└─ evaluation/
    └─ results_template.md
(optional: table for accuracy results)
```

Next Steps

- Build the labelled dataset.
- Write the classification prompt and run the evaluation.
- Report results against the success criteria above.

Prepared as part of the SWYNEX Technologies internship, Task 1.